TRABAJO PRÁCTICO OBLIGATORIO — PROGRAMACIÓN III

Informe parcial - Propuesta 2: Planificación de rutas y distribución de ayuda

Integrantes: Ovelar Evangelina, Peralta Luka
Primera entrega: 23 de octubre de 2026
Repositorio: https://github.com/eovelar/TPO-PROGRAMACION-III

1. Introducción y alcance

En este trabajo desarrollamos un sistema en Java para organizar la distribución de alimentos, medicamentos e insumos a distintos centros comunitarios. El programa trabaja con depósitos, centros, conexiones viales, vehículos, pedidos y lotes de mercadería. Para mantener el código ordenado, separamos las clases del modelo, los algoritmos y el menú de consola.
Para la primera entrega implementamos las tres funcionalidades obligatorias: ordenar y buscar pedidos, calcular la ruta más rápida y seleccionar cargas prioritarias. En las siguientes secciones explicamos cómo resolvimos cada problema, por qué elegimos los algoritmos y qué resultados obtuvimos en las pruebas.
Las tres funcionalidades quedaron integradas en el menú principal y el proyecto completo está disponible en la rama main de GitHub. La compilación finalizó sin errores y las pruebas realizadas dieron los resultados esperados.

2. Organización y consulta de pedidos

2.1. Datos y requisitos

Cada pedido tiene un ID, un centro de destino, una fecha límite, una prioridad y un peso total. El sistema permite ordenar por fecha o por prioridad y buscar un pedido por su ID. Cuando dos pedidos tienen la misma prioridad, se compara primero la fecha límite y después el ID menor. El ordenamiento y la búsqueda fueron implementados sin usar métodos de Java que resuelvan directamente esas operaciones.

2.2. Implementación

La clase Pedido guarda los datos de cada pedido y valida que sean correctos al crearlo. GestorPedidos administra los pedidos en un ArrayList, evita IDs repetidos y llama a los algoritmos de ordenamiento y búsqueda. Para ordenar sin cambiar la lista original, primero convierte los pedidos en un arreglo nuevo.
Para los ordenamientos usamos MergeSortPedidos, que permite ordenar por fecha, prioridad o ID. El método mergeSort divide el arreglo en dos partes de forma recursiva,

comparar determina qué pedido debe ir primero y merge une las mitades ya ordenadas. Para buscar por ID, GestorPedidos ordena una copia y llama a BusquedaBinariaPedidos, que va descartando la mitad del arreglo donde no puede estar el pedido.

2.3. Estrategias, justificación y alternativas

Elegimos Merge Sort porque utiliza Divide y Conquista: divide el arreglo en partes más pequeñas y luego las vuelve a unir en orden. Su complejidad es O(n log n), incluso en el peor caso, por lo que resulta conveniente si aumenta la cantidad de pedidos. También consideramos Bubble Sort, pero su complejidad $O(n^2)$ es menos eficiente para muchos elementos.
La búsqueda binaria necesita que los pedidos estén ordenados por ID. Compara el elemento del medio con el ID buscado y continúa en una sola mitad del arreglo. Una búsqueda lineal sería más simple y no necesitaría ordenar los datos, pero puede revisar todos los pedidos. La búsqueda binaria sola tiene complejidad O(log n); como nuestro método ordena una copia antes de buscar, el costo total de buscarPedidoPorId es O(n log n).

2.4. Complejidad

Ordenamiento por fecha, prioridad o ID: tiempo O(n log n) y espacio auxiliar O(n).
Búsqueda binaria sobre arreglo ya ordenado: tiempo O(log n) y espacio O(1) en su implementación iterativa.
Búsqueda mediante GestorPedidos (copia + ordenamiento por ID + búsqueda binaria): tiempo O(n log n) y espacio O(n).
Agregar pedido con verificación de duplicado utiliza una búsqueda que vuelve a ordenar: tiempo O(n log n) para n pedidos existentes.

2.5. Pruebas realizadas

Para los pedidos con ID 103 (20/10, prioridad 5), 104 (17/10, prioridad 3), 102 (18/10, prioridad 5) y 101 (18/10, prioridad 5), se obtuvo el orden por fecha 104 → 101 → 102 → 103 y el orden por prioridad 101 → 102 → 103 → 104. La búsqueda del ID 102 resultó satisfactoria; la consulta del ID 999 no encontró coincidencias. También se verificó el comportamiento ante un gestor vacío y el rechazo de registros con peso negativo, destino de tipo depósito, fecha nula o identificador duplicado. Los resultados observados coincidieron con los previstos.

3. Selección de cargas prioritarias

3.1. Datos y objetivo

En esta funcionalidad trabajamos con vehículos que tienen una capacidad máxima y con lotes que poseen un ID, un peso entero positivo y una prioridad no negativa. El

objetivo es elegir la combinación con mayor prioridad total sin superar la capacidad del vehículo. Los lotes no se pueden dividir ni cargar más de una vez. Este cálculo se hace por separado de las rutas.

3.2. Implementación

Las clases LoteMercaderia y Vehiculo guardan los datos de los lotes y del vehículo. También validan que los valores ingresados sean correctos. El algoritmo se encuentra en SeleccionCargas y muestra por consola los lotes elegidos, el peso utilizado, la prioridad total y la capacidad restante.
Para resolver la selección usamos el problema de mochila 0/1 con Programación Dinámica. Creamos una matriz dp de (n + 1) filas y (C + 1) columnas, donde n es la cantidad de lotes y C la capacidad del vehículo. El valor dp[i][j] indica la máxima prioridad posible usando los primeros i lotes con una capacidad j. Para cada lote comparamos dos opciones: no incluirlo o incluirlo si entra. La relación utilizada es dp[i][j] = max(dp[i−1][j], prioridad_i + dp[i−1][j−peso_i]). Al terminar, recorremos la matriz desde el final para saber qué lotes fueron seleccionados.

3.3. Estrategia y alternativas

Elegimos Programación Dinámica porque permite guardar resultados de combinaciones parciales y volver a usarlos, en lugar de calcularlos varias veces. Un método Greedy, como elegir primero el lote de mayor prioridad o el de mejor relación prioridad/peso, no siempre encuentra la mejor combinación cuando los lotes son indivisibles. También se podría usar Backtracking para probar combinaciones, pero en el peor caso tendría que explorar una cantidad exponencial de posibilidades.

3.4. Complejidad

Si n es la cantidad de lotes y C la capacidad máxima expresada en unidades enteras, el tiempo es O(n × C) y el espacio O(n × C) debido a la matriz. Es una complejidad seudopolinómica: depende del valor numérico de la capacidad, no solamente de su cantidad de dígitos.

3.5. Pruebas realizadas

Con un vehículo de capacidad 50 y lotes de datos (ID 1: peso 10, prioridad 30; ID 2: peso 20, prioridad 40; ID 3: peso 30, prioridad 50; ID 4: peso 40, prioridad 60), el algoritmo seleccionó los lotes 2 y 3. Se obtuvo un peso total de 50, prioridad acumulada de 90 y capacidad restante igual a cero.
Se verificaron, además, un vehículo de capacidad 5 en el que no podía incorporarse ningún lote; la ausencia de lotes disponibles; el rechazo de pesos o capacidades inválidos; y una situación de empate en la que se conservó una solución óptima válida sin necesidad de completar toda la capacidad. Los casos ejecutados finalizaron sin errores.

4. Determinación de la ruta de menor tiempo

4.1. Datos y restricciones

La red vial está formada por depósitos, centros comunitarios y conexiones con un sentido de circulación y un tiempo positivo en minutos. Algunas conexiones pueden estar deshabilitadas. Al indicar un origen y un destino, el programa busca el recorrido de menor tiempo y muestra las ubicaciones por las que pasa. Si se deshabilita una conexión, se puede volver a calcular la ruta, y si no existe un camino posible, el programa lo informa.

4.2. Estructura del módulo

Para trabajar con rutas usamos las clases ConexionVial, RedVial, Dijkstra, ResultadoRuta y PruebaRutas. ConexionVial representa cada conexión y su estado; RedVial guarda las ubicaciones y conexiones; Dijkstra calcula el camino más rápido; ResultadoRuta guarda el recorrido, el tiempo y las decisiones tomadas; y PruebaRutas permite comprobar distintos casos.

4.3. Selección y fundamento de la estrategia

Elegimos Dijkstra porque permite obtener el camino más corto cuando los tiempos de las conexiones son positivos. Aplica la estrategia Greedy: en cada paso selecciona la ubicación no visitada con menor tiempo acumulado y revisa si puede mejorar las distancias hacia sus vecinos. También guarda desde qué ubicación se llegó a cada una, para después reconstruir el recorrido.
Una alternativa sería recorrer todos los caminos posibles y quedarse con el más corto, pero eso puede requerir muchas combinaciones. También consideramos Floyd–Warshall, que sirve para obtener caminos mínimos entre todos los pares de ubicaciones. Para esta primera etapa elegimos Dijkstra porque las consultas parten de un origen y un destino concretos; Floyd–Warshall queda como opción para la funcionalidad A1.

4.4. Complejidad

En nuestro código, Dijkstra busca el nodo de menor distancia recorriendo una lista de ubicaciones pendientes. Además, RedVial recorre la lista de conexiones para obtener las que salen de cada ubicación. Por eso, tomando V como la cantidad de ubicaciones y E como la cantidad de conexiones, la complejidad temporal se puede acotar por $O(V^2 + V \cdot E)$. No usamos una cola de prioridad. La memoria auxiliar para distancias, predecesores y nodos pendientes es O(V), sin contar las decisiones que se guardan para mostrarlas ni las listas temporales de conexiones.

4.5. Casos de prueba y resultados obtenidos

Caso 1 — Ruta de menor tiempo: desde Depósito Central hasta Centro Este se obtuvo el recorrido Depósito Central → Centro Norte → Centro Este, con un tiempo total de 15 minutos. Resultado satisfactorio.
Caso 2 — Modificación de la red: al deshabilitar la conexión Centro Norte → Centro Este, el sistema recalculó el recorrido Depósito Central → Centro Sur → Centro Este, con una duración de 23 minutos. Resultado satisfactorio.
Caso 3 — Destino inaccesible: desde Centro Este hacia Depósito Central el sistema informó que no existía una ruta disponible. Resultado satisfactorio.
Caso 4 — Origen y destino coincidentes: el algoritmo devolvió el Depósito Central como único punto del recorrido y un tiempo total de 0 minutos. Resultado satisfactorio.
Comprobación complementaria: las conexiones se consideran dirigidas; por ello, la existencia de una conexión de origen a destino no implica automáticamente el recorrido inverso.
Trazabilidad: en las pruebas se registraron las ubicaciones procesadas, las actualizaciones de distancias y la identificación del destino alcanzado o inaccesible. Estas salidas permiten seguir las decisiones del algoritmo.
El archivo PruebaRutas concluyó con cuatro pruebas y cero errores, según la ejecución realizada. Asimismo, la consulta de origen 1 a destino 4 y el recálculo posterior a deshabilitar la conexión 2 → 4 se comprobaron desde el menú de la aplicación integrada.

5. Integración y pruebas generales

5.1. Interfaz de consola y organización

El archivo Main.java contiene el menú que reúne las tres funcionalidades. Desde allí se pueden agregar, ordenar y buscar pedidos; consultar rutas, deshabilitar o habilitar conexiones; e ingresar lotes y la capacidad del vehículo para calcular una carga. Los algoritmos están en clases separadas para no mezclar la lógica del programa con la entrada y salida de datos.

5.2. Pruebas de los módulos ya ejecutadas

Probamos los ordenamientos de pedidos, las búsquedas con IDs existentes e inexistentes, una lista vacía y el rechazo de pedidos inválidos o duplicados. Para cargas probamos la selección óptima, un arreglo sin lotes, casos donde no entra ningún lote, empates y valores inválidos. Al ejecutar TestAlgoritmos.java obtuvimos 10 pruebas correctas y ninguna fallida.

5.3. Integración de los módulos

Unimos el trabajo de ambos integrantes en la rama main y comprobamos que el proyecto compile. Desde el menú de rutas calculamos el recorrido Depósito Central →

Centro Norte → Centro Este, con un tiempo de 15 minutos. Luego deshabilitamos la conexión Centro Norte → Centro Este y repetimos la consulta. El programa encontró la ruta por Centro Sur, con un tiempo total de 23 minutos. Los módulos de pedidos y cargas también se habían probado desde el menú, antes de unir ambas ramas.
Como control final, queda realizar una prueba de las tres opciones en una misma ejecución y guardar los resultados para la entrega.

6. Funcionalidades adicionales elegidas para la segunda etapa

A1. Consultas de rutas entre todos los centros: preparar y conservar tiempos mínimos y recorridos entre pares de ubicaciones.
B1. Recorrido con múltiples entregas: determinar el orden de visita de menor tiempo, saliendo del depósito y regresando a él, con un máximo inicial de ocho centros.
B3. Rutas alternativas con restricciones: encontrar todos los recorridos válidos entre origen y destino, respetando el tiempo máximo y ubicaciones prohibidas, sin repetir ubicaciones.
Estas funcionalidades se identifican en esta primera entrega; no están implementadas todavía.

7. Conclusiones y estado de la primera entrega

En esta primera entrega completamos la organización y búsqueda de pedidos, el cálculo de rutas de menor tiempo y la selección de cargas prioritarias. Para resolver estos problemas usamos Divide y Conquista, Greedy y Programación Dinámica, y comprobamos los resultados con distintos casos de prueba. Los módulos quedaron integrados en main. Para la segunda entrega, prevista para el 6 de noviembre de 2026, elegimos A1, B1 y B3, donde también trabajaremos con Backtracking. Nos queda realizar una última prueba general antes de presentar el proyecto.